

机器视觉实验报告

专业 智能科学与技术 姓名 史峰源 学号 2412526

基于 ResNet18 迁移学习的手势姿态识别

1 实验目的

- 完成六类手势姿态图像分类任务，识别类别包括 A、B、C、Five、Point、V。
- 使用 ResNet18 预训练模型进行迁移学习，构建适用于小规模手势数据集的分类模型。
- 实现分层数据划分，使训练集和验证集在各类别上保持一致比例。
- 统一验证与推理阶段的确定性预处理流程，保证实验结果具有可复现性。
- 通过数据分布图、样例图和训练指标对实验过程进行辅助分析。

2 实验环境与数据集

2.1 实验环境

本实验使用 Python 与 PyTorch 完成模型训练和推理。主要依赖包括 torch、torchvision、pillow、tqdm、scikit-learn 和 matplotlib。训练时优先使用 CUDA 设备；本机实验环境可识别 NVIDIA GeForce RTX 4050 Laptop GPU，因此训练过程使用 GPU 加速。

表 1: 主要实验配置

项目	配置
深度学习框架	PyTorch
基础模型	ResNet18
预训练权重	ImageNet 预训练权重
输入尺寸	224×224
批大小	32
训练轮数	30
学习率	1×10^{-4}
优化器	AdamW
学习率调度	CosineAnnealingLR
损失函数	CrossEntropyLoss, label smoothing 为 0.1

2.2 数据集组成与划分

数据集共包含六个类别：A、B、C、Five、Point、V。原始数据按照 ImageFolder 格式组织，即每个类别对应一个独立文件夹。实验采用分层划分策略，将每一类图像按约 80% 和 20% 分别划入训练集和验证集，从而使验证集能够保持与原始数据一致的类别比例。

表 2: 各类别图像数量统计

类别	原始数据	训练集	验证集
A	1327	1062	265
B	508	406	102
C	604	483	121
Five	707	566	141
Point	1394	1115	279
V	500	400	100
合计	5040	4032	1008

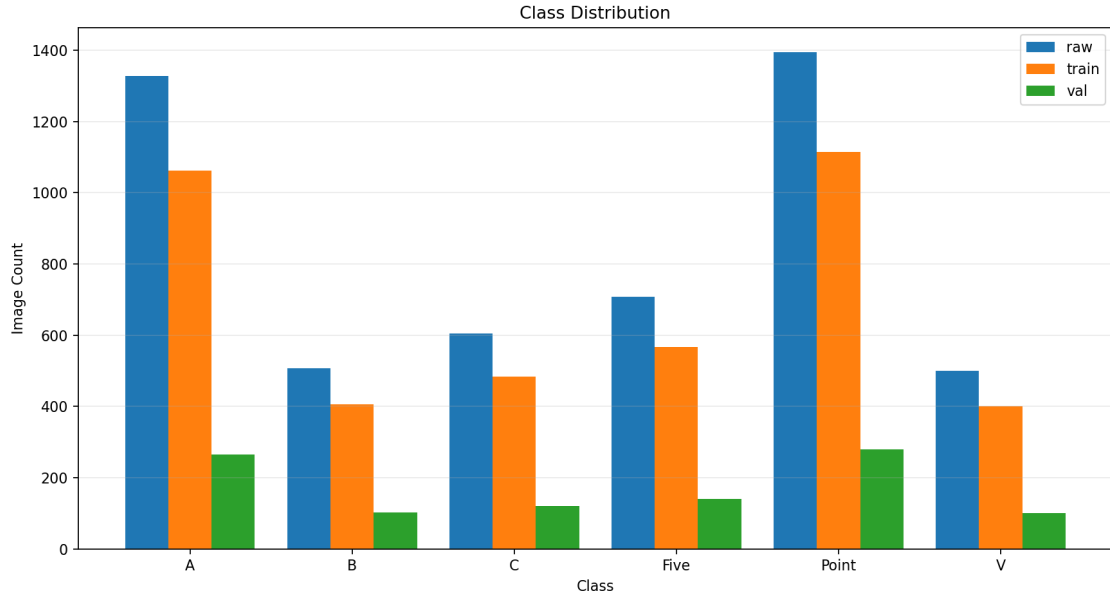


图 1: 原始数据、训练集和验证集类别数量分布

3 实验原理与方法

3.1 迁移学习模型

ResNet 通过残差连接缓解深层神经网络训练中的梯度退化问题，其基本思想是学习残差映射：

$$\mathcal{H}(x) = \mathcal{F}(x) + x \quad (1)$$

其中 x 为输入特征， $\mathcal{F}(x)$ 为残差分支学习到的变换。相较于从零开始训练卷积神经网络，使用 ImageNet 预训练的 ResNet18 能够利用已有的通用视觉特征，如边缘、纹理和局部形状，从而提高小规模图像分类任务的训练效率与稳定性。

本实验保留 ResNet18 的卷积特征提取部分，并将原始全连接分类层替换为适用于六分类任务的新分类头。分类头包含 Dropout、线性层、ReLU 激活和最终输出层，用于降低过拟合风险并输出六个类别的 logits。

3.2 数据预处理

训练阶段使用随机增强以提升模型对尺度、平移、旋转和光照变化的适应能力。具体包括随机裁剪、水平翻转、随机旋转、颜色扰动和仿射变换。验证阶段不再使用随机操作，而是采用确定性的 `Resize(256)` 和

CenterCrop(224)，保证同一张图像每次评估时输入一致。

训练阶段预处理流程如下：

```
transforms.Resize(256)
transforms.RandomResizedCrop(224, scale=(0.75, 1.0))
transforms.RandomHorizontalFlip(p=0.5)
transforms.RandomRotation(15)
transforms.ColorJitter(
    brightness=0.25,
    contrast=0.25,
    saturation=0.25,
    hue=0.05
)
transforms.RandomAffine(
    degrees=0,
    translate=(0.08, 0.08),
    scale=(0.9, 1.1)
)
transforms.ToTensor()
transforms.Normalize(mean=[0.485, 0.456, 0.406],
                      std=[0.229, 0.224, 0.225])
```

验证阶段预处理流程如下：

```
transforms.Resize(256)
transforms.CenterCrop(224)
transforms.ToTensor()
transforms.Normalize(mean=[0.485, 0.456, 0.406],
                      std=[0.229, 0.224, 0.225])
```

3.3 训练策略

模型训练使用交叉熵损失函数，并加入 label smoothing 以缓解模型过度自信的问题。优化器选用 AdamW，学习率为 1×10^{-4} ，权重衰减为 1×10^{-4} 。学习率调度器采用余弦退火策略，在 30 个 epoch 内逐步调整学习率。

训练过程中记录每轮训练损失、训练准确率、验证损失和验证准确率。当验证准确率超过历史最佳值时，保存当前模型权重及类别映射信息。保存的 checkpoint 包含 model_state_dict、class_to_idx、classes 和 best_acc，从而避免推理阶段出现类别顺序不一致的问题。

4 程序代码

以下给出本实验中与数据划分、模型构建、训练保存和推理标签映射相关的核心代码片段。完整代码见项目目录中的 *Python* 脚本。

4.1 数据分层划分

数据划分脚本按照类别文件夹分别打乱并拆分图像，保证每个类别在训练集和验证集中都保持近似 80/20 的比例。

```
def split_class_files(files, val_ratio, rng):
```

```

files = list(files)
rng.shuffle(files)

if len(files) <= 1:
    return files, []

val_count = round(len(files) * val_ratio)
val_count = min(max(1, val_count), len(files) - 1)

val_files = files[:val_count]
train_files = files[val_count:]

return train_files, val_files

```

4.2 模型构建

模型基于 torchvision 提供的 ResNet18，并替换最后的分类层以适配六分类任务。

```

def build_resnet18(num_classes=6, pretrained=True):
    if pretrained:
        weights = models.ResNet18_Weights.DEFAULT
    else:
        weights = None

    model = models.resnet18(weights=weights)
    in_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(0.4),
        nn.Linear(in_features, 256),
        nn.ReLU(inplace=True),
        nn.Dropout(0.3),
        nn.Linear(256, num_classes)
    )
    return model

```

4.3 训练与 checkpoint 保存

训练脚本使用验证准确率选择最优模型，并将类别映射与模型权重一起保存。

```

if val_acc > best_acc:
    best_acc = val_acc
    best_model_wts = copy.deepcopy(model.state_dict())

    torch.save({
        "model_state_dict": best_model_wts,
        "class_to_idx": train_dataset.class_to_idx,
        "classes": train_dataset.classes,
        "best_acc": best_acc
    }, "checkpoints/best_model.pth")

```

4.4 推理阶段类别映射

推理阶段不再硬编码类别顺序，而是从 checkpoint 读取训练时保存的类别映射。

```
def get_idx_to_class(checkpoint):
    if "class_to_idx" in checkpoint:
        idx_to_class = {
            int(index): class_name
            for class_name, index in checkpoint["class_to_idx"].items()
        }
    elif "classes" in checkpoint:
        idx_to_class = {
            index: class_name
            for index, class_name in enumerate(checkpoint["classes"])
        }
    else:
        raise ValueError("Checkpoint must include class metadata.")

    return idx_to_class
```

5 实验结果与分析

5.1 数据可视化结果

Figure 1 展示了各类别图像数量分布。可以看出，A 和 Point 类别样本数较多，而 B 和 V 类别样本数较少，数据集存在一定类别不均衡。通过分层划分，训练集和验证集保留了原始数据的类别比例，使验证结果更能反映同源数据分布下的模型表现。

Samples: data\raw

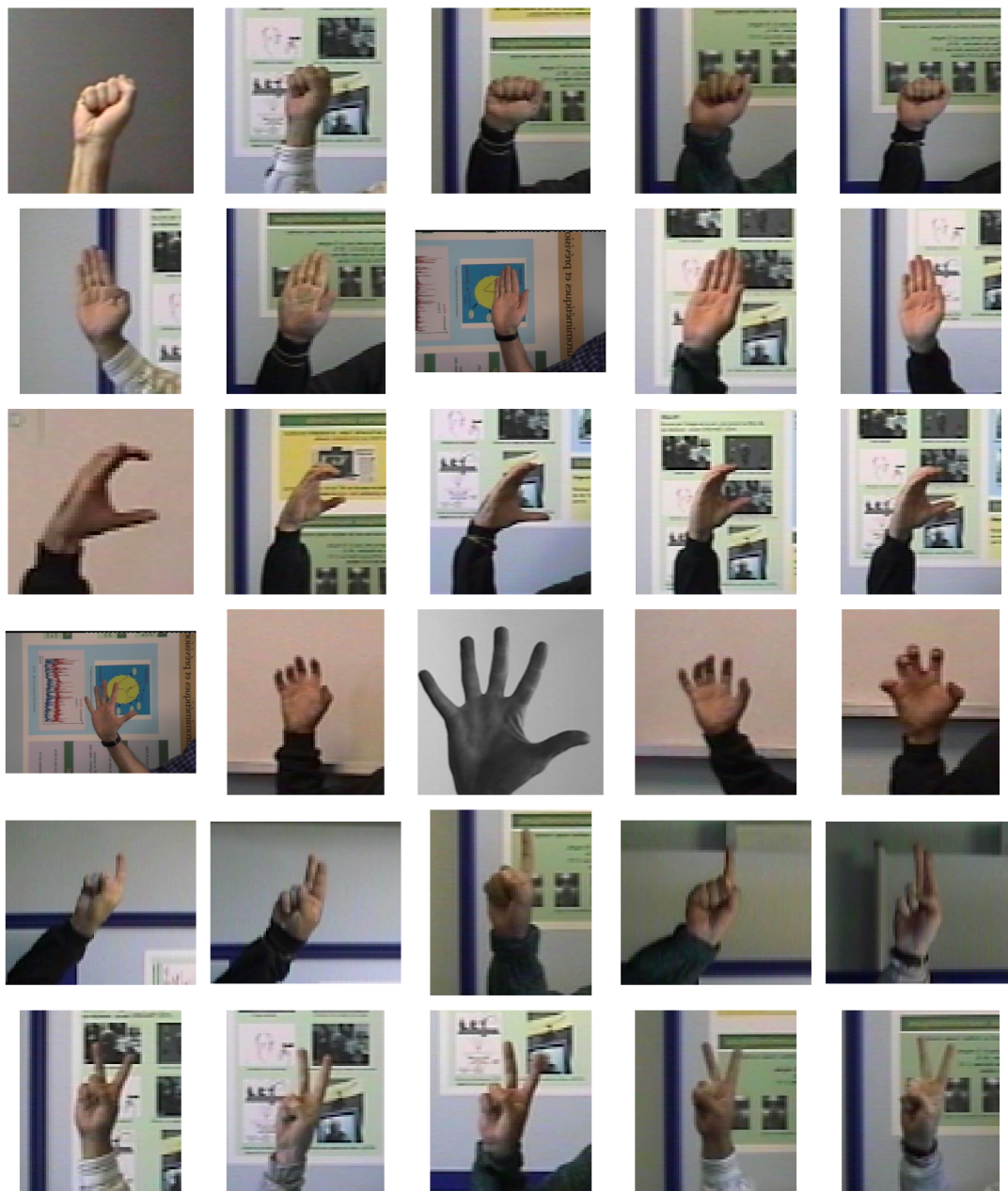


图 2: 原始数据集中各类别样例

Samples: data\train

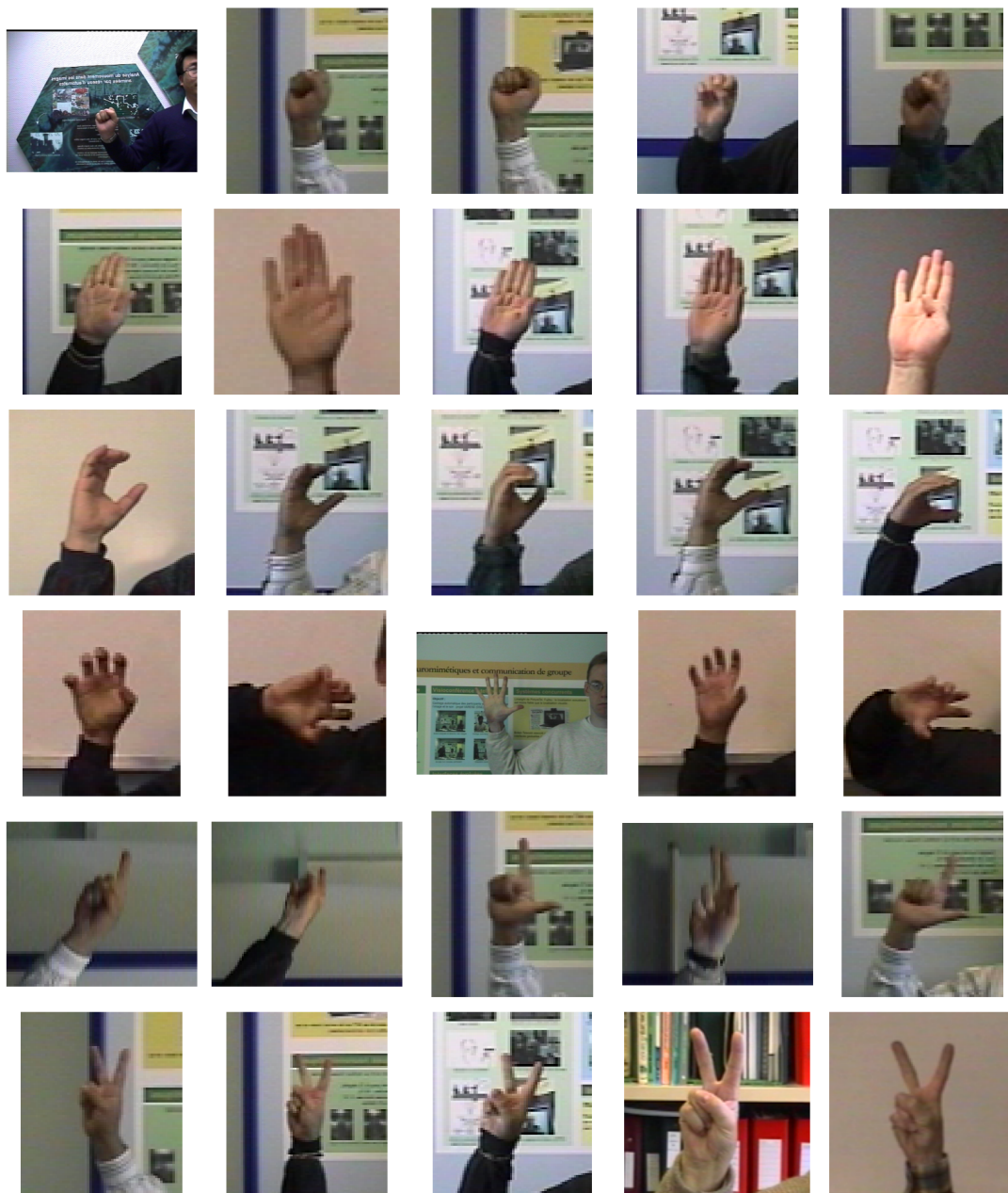


图 3: 训练集中各类别样例

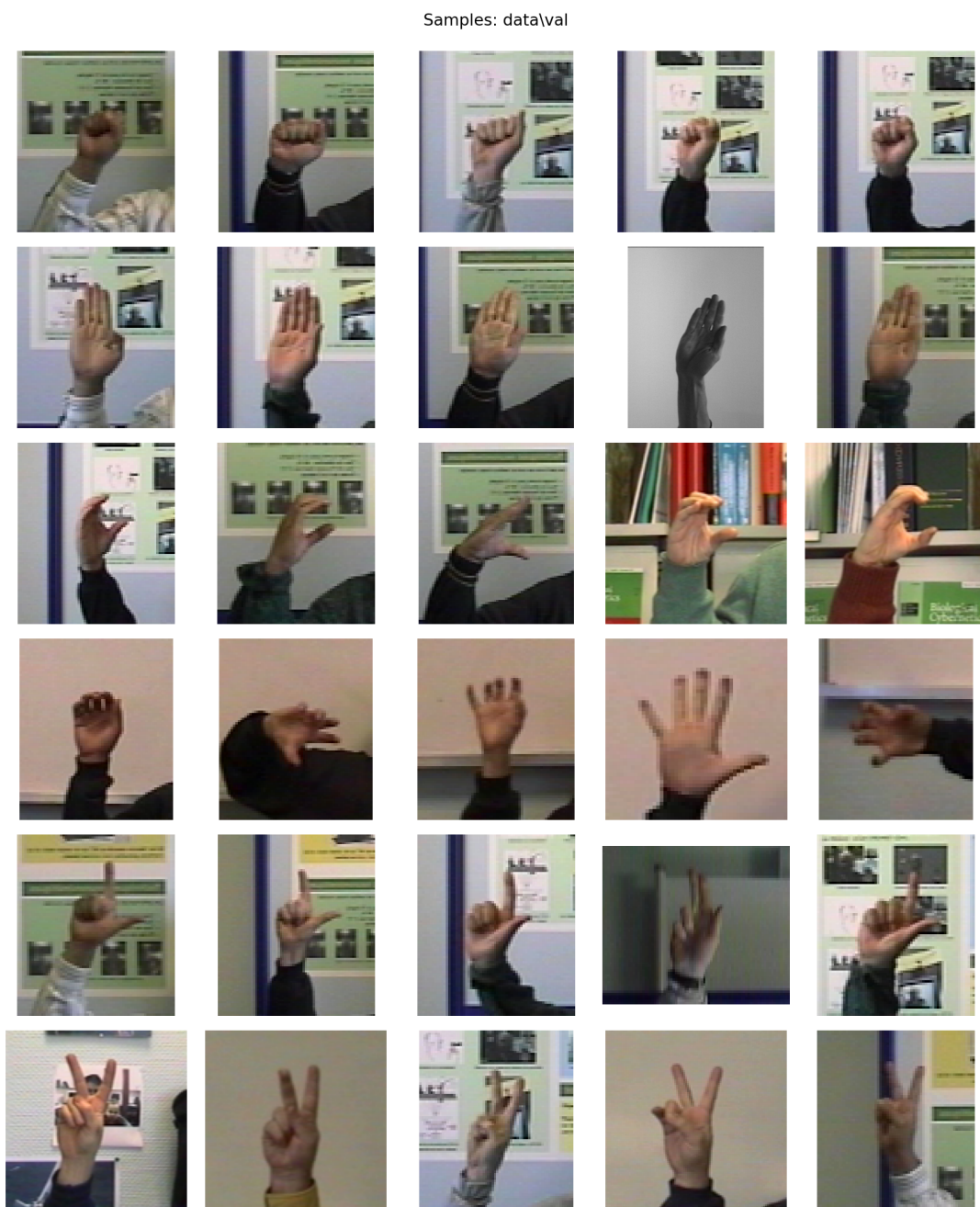


图 4: 验证集中各类别样例

5.2 训练结果

模型训练共进行 30 个 epoch。训练过程中，模型在验证集上取得的最佳准确率为：

$$\text{Best Val Acc} = 1.0000 \quad (2)$$

即在当前分层验证集上达到 100% 的分类准确率。最佳模型保存为 `checkpoints/best_model.pth`。从实验结果看，迁移学习能够在较短训练轮数内充分利用预训练特征，并快速适应当前六类手势分类任务。

为进一步观察最终模型在课程验证集上的分类情况，使用保存的最佳 checkpoint 对 `data/val` 中的 1008 张验证图像重新进行推理，并绘制混淆矩阵和各类别准确率图。结果显示，六个类别的预测均集中在混淆矩阵对角线上，未出现跨类别误判；各类别准确率均为 1.0000。

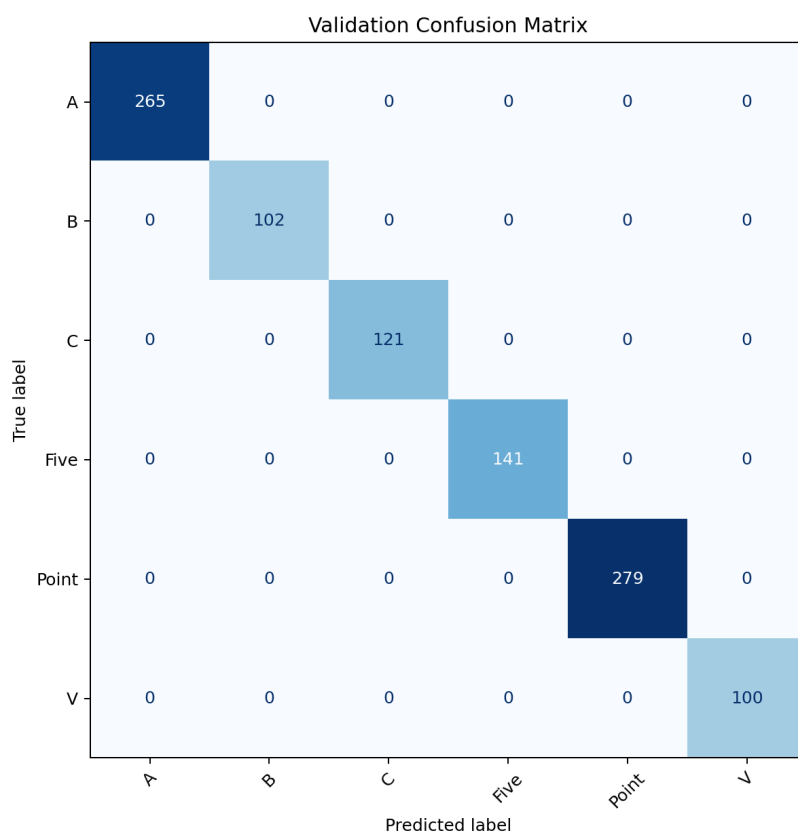


图 5: 最终模型在验证集上的混淆矩阵

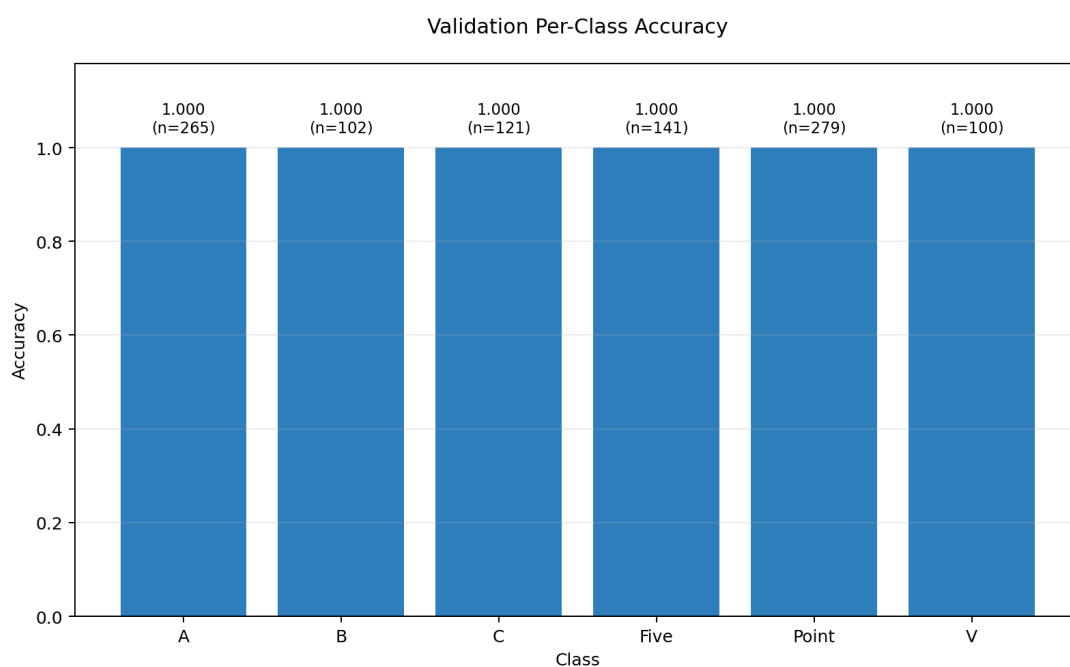


图 6: 最终模型在验证集上的各类别准确率

需要注意的是，验证集由同一原始数据集分层划分得到，因此该指标主要反映模型在同源数据分布下的识别能力。为了保证实验目标明确，本报告仅分析课程数据集划分得到的训练与验证结果。

5.3 结果分析

从数据分布和样例图可以观察到，不同类别在手形、手指数量和姿态方向上具有较明显差异，这有利于卷积神经网络进行特征学习。训练阶段引入随机裁剪、旋转、颜色扰动和仿射变换，可以减少模型对固定位置和固定光照的依赖。验证阶段使用确定性中心裁剪，使每次验证输入一致，保证实验指标可复现。

此外，checkpoint 中保存类别映射是本实验中的重要工程改进。若仅在推理代码中手动写入类别列表，一旦训练数据目录顺序或类别名称发生变化，模型输出下标与类别名可能不一致。将 `class_to_idx` 与模型权重一起保存，可以保证推理阶段与训练阶段使用完全相同的标签语义。

6 实验结论

本实验完成了基于 ResNet18 迁移学习的六类手势姿态识别模型。通过分层划分数据集、统一验证与推理预处理、保存类别映射和可视化数据分布，项目形成了较完整的训练与分析流程。实验结果表明，模型在课程数据集划分得到的验证集上达到 100% 准确率，说明迁移学习方法能够有效适应该手势分类任务。

后续若继续改进，可在不改变当前最佳模型的前提下，增加更细粒度的类别混淆分析、训练日志曲线记录和多次随机种子实验，以进一步评估模型稳定性。